Computer Vision Practice Programs

10 runnable exercises walking through the image processing pipeline — from raw pixels (**low-level vision**) to segmentation and features (**mid-level vision**) to recognition and decision-making (**high-level vision**).

Every section synthesizes its own test image with OpenCV/NumPy, so the whole notebook runs top-to-bottom with no external files needed. Images are displayed inline via matplotlib.

Run: `Kernel -> Restart & Run All`

Setup

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def show(images, titles=None, cmap=None, figsize=None):
    """Display one or more images side by side. Converts BGR->RGB for color
    if not isinstance(images, list):
        images = [images]
    if titles is None:
        titles = [None] * len(images)
    n = len(images)
    figsize = figsize or (5 * n, 4)
    fig, axes = plt.subplots(1, n, figsize=figsize)
    if n == 1:
        axes = [axes]
    for ax, img, title in zip(axes, images, titles):
        if img.ndim == 3:
            ax.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        else:
            ax.imshow(img, cmap=cmap or 'gray')
        if title:
            ax.set_title(title)
        ax.axis('off')
    plt.tight_layout()
    plt.show()

print("OpenCV version:", cv2.__version__)
```

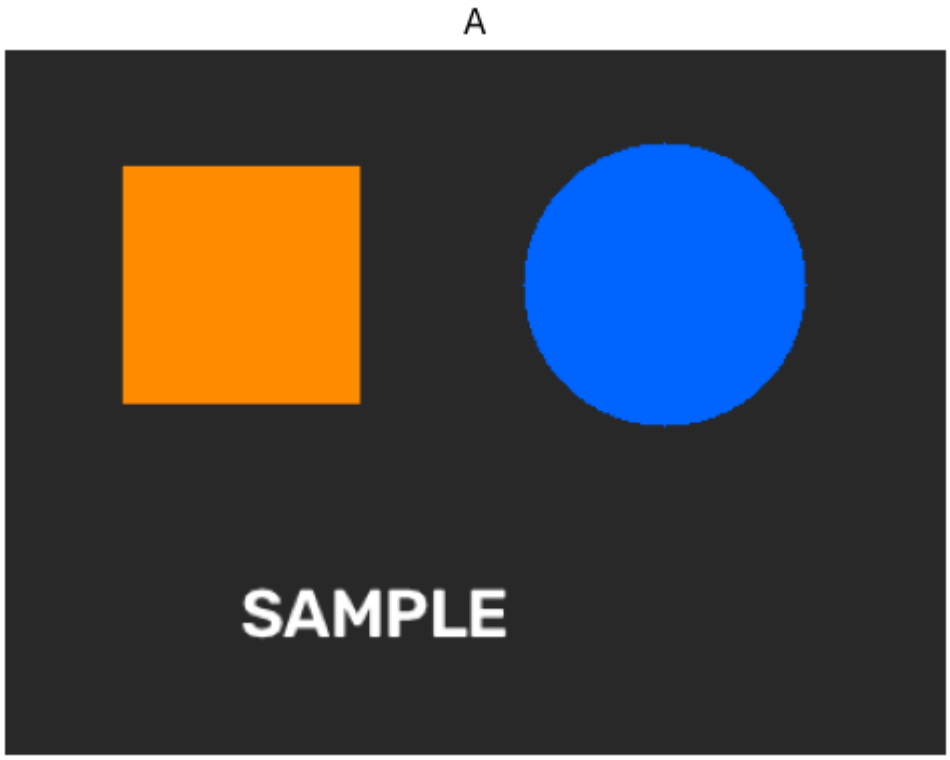
```
OpenCV version: 5.0.0
```

1. Image Acquisition & Basic Properties *(Low-Level Vision)*

Understand how a digital image is represented as raw pixel data. We synthesize an image (simulating "acquisition") so the exercise is self-contained — swap this for `cv2.imread('photo.jpg')` or `cv2.VideoCapture(0)` to acquire from a file or webcam.

```
def create_sample_image():
    img = np.full((300, 400, 3), (40, 40, 40), dtype=np.uint8)
    cv2.rectangle(img, (50, 50), (150, 150), (0, 140, 255), -1)
    cv2.circle(img, (280, 100), 60, (255, 100, 0), -1)
    cv2.putText(img, "SAMPLE", (100, 250), cv2.FONT_HERSHEY_SIMPLEX, 1, (255
    return img

image = create_sample_image()

print("Image shape (H, W, C):", image.shape)
print("Data type:", image.dtype)
print("Total pixels:", image.shape[0] * image.shape[1])
print("Min / Max pixel value:", image.min(), image.max())

b, g, r = cv2.split(image)
print(f"Mean intensity - B: {b.mean():.2f}, G: {g.mean():.2f}, R: {r.mean():

show(image, "Acquired Image")
```

```
Image shape (H, W, C): (300, 400, 3)
Data type: uint8
Total pixels: 120000
Min / Max pixel value: 0 255
Mean intensity - B: 58.90, G: 56.22, R: 56.59
```



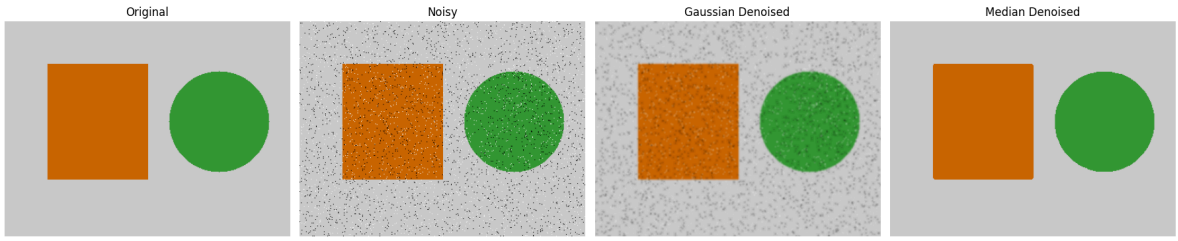
---

2. Preprocessing — Noise Reduction & Normalization *(Low-Level Vision)*

Clean up raw pixel data before further analysis: adding synthetic noise, then comparing Gaussian vs. median blur, plus contrast normalization via histogram equalization.

```
def create_sample_image_2():
    img = np.full((300, 400, 3), (200, 200, 200), dtype=np.uint8)
    cv2.rectangle(img, (60, 60), (200, 220), (0, 100, 200), -1)
    cv2.circle(img, (300, 140), 70, (50, 150, 50), -1)
    return img

def add_salt_and_pepper_noise(img, amount=0.02):
    noisy = img.copy()
    h, w = img.shape[:2]
    n_pixels = int(amount * h * w)
    ys, xs = np.random.randint(0, h, n_pixels), np.random.randint(0, w, n_pi
    noisy[ys, xs] = 255
    ys, xs = np.random.randint(0, h, n_pixels), np.random.randint(0, w, n_pi
    noisy[ys, xs] = 0
    return noisy
```

```
original = create_sample_image_2()
noisy = add_salt_and_pepper_noise(original)
gaussian_denoised = cv2.GaussianBlur(noisy, (5, 5), 0)
median_denoised = cv2.medianBlur(noisy, 5)

gray = cv2.cvtColor(original, cv2.COLOR_BGR2GRAY)
equalized = cv2.equalizeHist(gray)

show([original, noisy, gaussian_denoised, median_denoised],
     ["Original", "Noisy", "Gaussian Denoised", "Median Denoised"], figsize=
show(equalized, "Histogram Equalized (Grayscale)")

print("Median blur removes salt & pepper noise far better than Gaussian,")
print("because it replaces each pixel with the median of its neighborhood,")
print("which is robust to extreme outlier pixel values.")
```



H

```
Median blur removes salt & pepper noise far better than Gaussian,
because it replaces each pixel with the median of its neighborhood,
which is robust to extreme outlier pixel values.
```

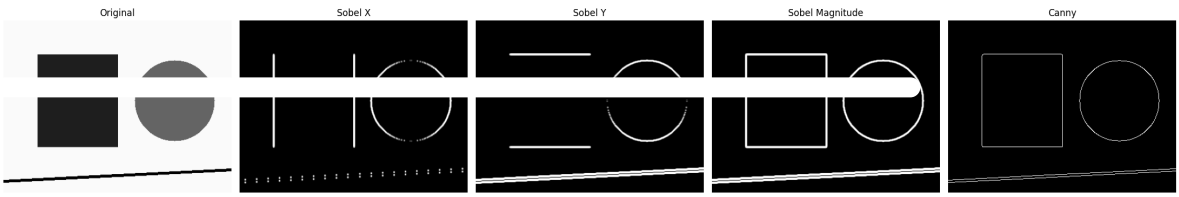
---

3. Edge Detection — Sobel & Canny *(Low-Level Vision)*

Find boundaries between regions purely from intensity gradients — no notion yet of "what" the shapes are.

```
def create_sample_image_3():
    img = np.full((300, 400, 3), (250, 250, 250), dtype=np.uint8)
```

```
    cv2.rectangle(img, (60, 60), (200, 220), (30, 30, 30), -1)
    cv2.circle(img, (300, 140), 70, (100, 100, 100), -1)
    cv2.line(img, (0, 280), (400, 260), (0, 0, 0), 4)
    return img

img3 = create_sample_image_3()
gray3 = cv2.cvtColor(img3, cv2.COLOR_BGR2GRAY)
gray3 = cv2.GaussianBlur(gray3, (3, 3), 0)

sobel_x = cv2.Sobel(gray3, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray3, cv2.CV_64F, 0, 1, ksize=3)
sobel_mag = cv2.convertScaleAbs(cv2.magnitude(sobel_x, sobel_y))
canny_edges = cv2.Canny(gray3, 50, 150)

show([img3, cv2.convertScaleAbs(sobel_x), cv2.convertScaleAbs(sobel_y), sobe
     ["Original", "Sobel X", "Sobel Y", "Sobel Magnitude", "Canny"], figsize

print("Sobel-X highlights vertical edges, Sobel-Y highlights horizontal edge
print("Canny combines both directions plus thresholding into clean, thin edg
print("Try it yourself: change Canny's threshold1/threshold2 and re-run.")
```




```
Sobel-X highlights vertical edges, Sobel-Y highlights horizontal edges.
Canny combines both directions plus thresholding into clean, thin edges.
Try it yourself: change Canny's threshold1/threshold2 and re-run.
```

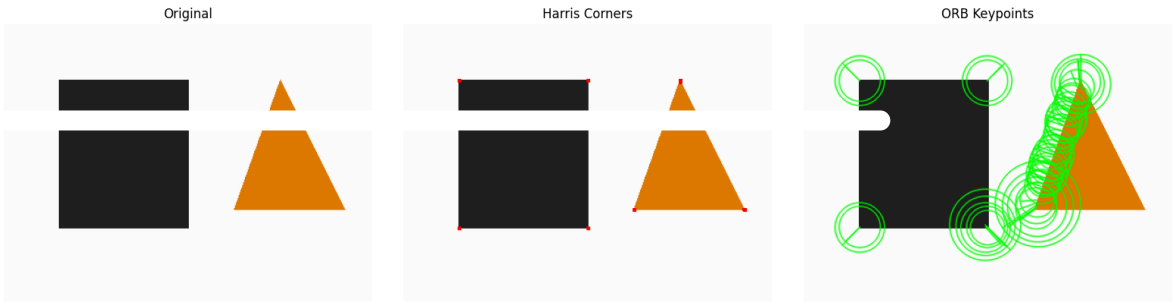
---

4. Feature Extraction — Corners & Keypoints *(Mid-Level Vision)*

Move beyond raw edges to distinctive, repeatable structures (corners, keypoints) that can be matched or tracked across images.

```
def create_sample_image_4():
    img = np.full((300, 400, 3), (250, 250, 250), dtype=np.uint8)
    cv2.rectangle(img, (60, 60), (200, 220), (30, 30, 30), -1)
```

```
    pts = np.array([[300, 60], [370, 200], [250, 200]], np.int32)
    cv2.fillPoly(img, [pts], (0, 120, 220))
    return img

img4 = create_sample_image_4()
gray4 = cv2.cvtColor(img4, cv2.COLOR_BGR2GRAY)

# Harris corners
harris = cv2.cornerHarris(np.float32(gray4), blockSize=2, ksize=3, k=0.04)
harris = cv2.dilate(harris, None)
harris_vis = img4.copy()
harris_vis[harris > 0.01 * harris.max()] = [0, 0, 255]

# ORB keypoints
orb = cv2.ORB_create(nfeatures=200)
keypoints, descriptors = orb.detectAndCompute(gray4, None)
orb_vis = cv2.drawKeypoints(img4, keypoints, None, color=(0, 255, 0),
                            flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINT

show([img4, harris_vis, orb_vis], ["Original", "Harris Corners", "ORB Keypoi

print(f"ORB found {len(keypoints)} keypoints.")
if descriptors is not None:
    print(f"Each keypoint has a {descriptors.shape[1]}-dim descriptor, used
    print("the same point across different images.")
```



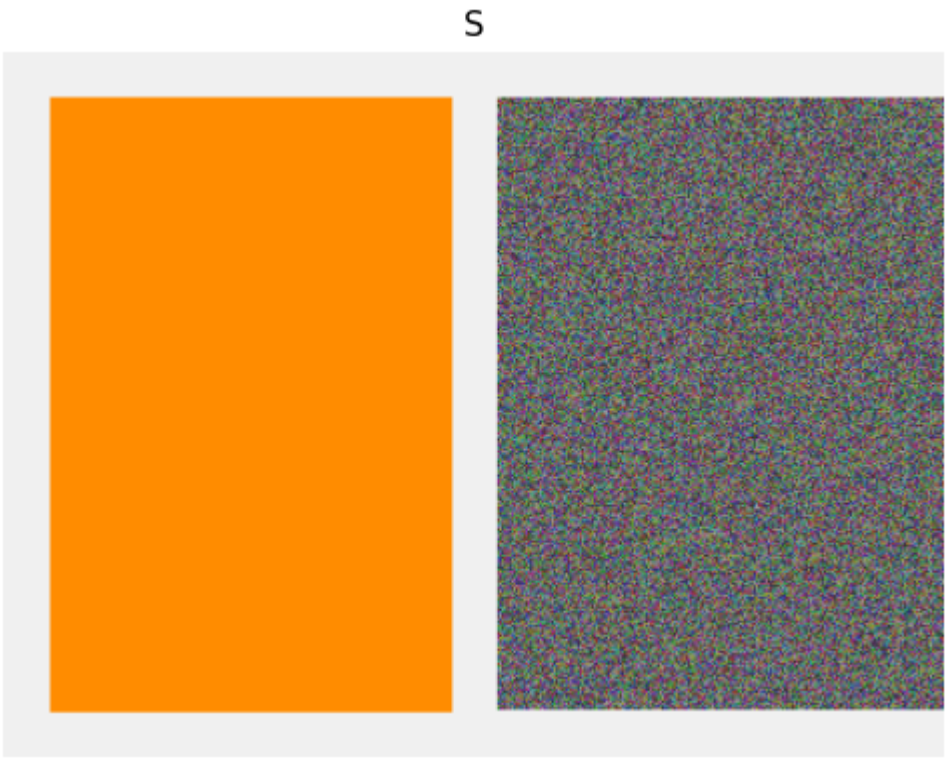

```
ORB found 42 keypoints.
Each keypoint has a 32-dim descriptor, used to match
the same point across different images.
```

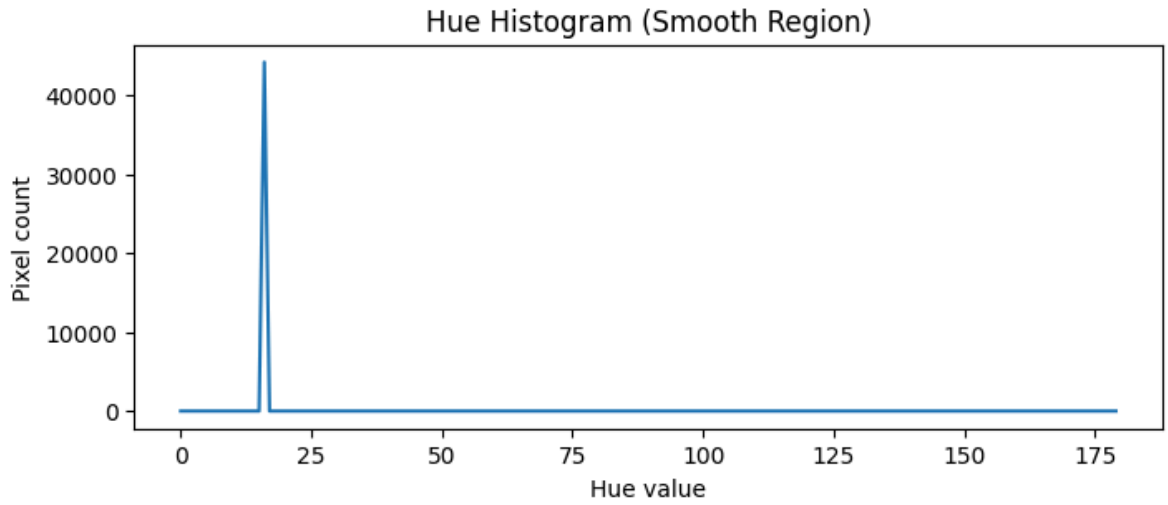
5. Color & Texture Analysis *(Mid-Level Vision)*

Extract pixel-level statistical properties describing regions — color distributions and texture "roughness" — without yet naming the object.

```
def create_sample_image_5():
    img = np.full((300, 400, 3), (240, 240, 240), dtype=np.uint8)
    cv2.rectangle(img, (20, 20), (190, 280), (0, 140, 255), -1)
    noise = np.random.randint(0, 255, (260, 190, 3), dtype=np.uint8)
    img[20:280, 210:400] = cv2.addWeighted(
        np.full((260, 190, 3), (80, 80, 80), dtype=np.uint8), 0.5, noise, 0.
    return img

def texture_energy(gray_region):
    return cv2.Laplacian(gray_region, cv2.CV_64F).var()

img5 = create_sample_image_5()
hsv5 = cv2.cvtColor(img5, cv2.COLOR_BGR2HSV)
gray5 = cv2.cvtColor(img5, cv2.COLOR_BGR2GRAY)

smooth_region, textured_region = img5[20:280, 20:190], img5[20:280, 210:400]
smooth_gray, textured_gray = gray5[20:280, 20:190], gray5[20:280, 210:400]

for name, region in [("Smooth region", smooth_region), ("Textured region", t
    mean_bgr = region.reshape(-1, 3).mean(axis=0)
    print(f"{name}: mean BGR = ({mean_bgr[0]:.1f}, {mean_bgr[1]:.1f}, {mean_

print(f"\nTexture energy (Laplacian variance):")
print(f"  Smooth region:   {texture_energy(smooth_gray):.2f}  (low = flat/un
print(f"  Textured region: {texture_energy(textured_gray):.2f}  (high = roug

hue_hist = cv2.calcHist([hsv5[20:280, 20:190]], [0], None, [180], [0, 180])
show(img5, "Smooth (left) vs Textured (right) Regions")

plt.figure(figsize=(8, 3))
plt.plot(hue_hist)
plt.title("Hue Histogram (Smooth Region)")
plt.xlabel("Hue value")
plt.ylabel("Pixel count")
plt.show()
```

```
Smooth region: mean BGR = (0.0, 140.0, 255.0)
Textured region: mean BGR = (103.3, 103.7, 103.7)

Texture energy (Laplacian variance):
  Smooth region:   0.00  (low = flat/uniform)
  Textured region: 12179.21  (high = rough/noisy)
```





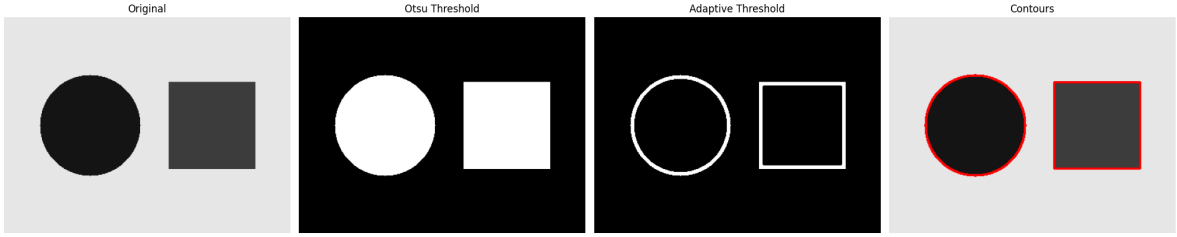
---

6. Segmentation — Thresholding & Contours *(Mid-Level Vision)*

Group pixels into meaningful regions — separating foreground from background — as an intermediate representation before semantic labeling.

```
def create_sample_image_6():
    img = np.full((300, 400, 3), (230, 230, 230), dtype=np.uint8)
    cv2.circle(img, (120, 150), 70, (20, 20, 20), -1)
    cv2.rectangle(img, (230, 90), (350, 210), (60, 60, 60), -1)
    return img

img6 = create_sample_image_6()
gray6 = cv2.cvtColor(img6, cv2.COLOR_BGR2GRAY)

_, otsu_thresh = cv2.threshold(gray6, 0, 255, cv2.THRESH_BINARY_INV + cv2.TH
adaptive_thresh = cv2.adaptiveThreshold(gray6, 255, cv2.ADAPTIVE_THRESH_GAUS
                                        cv2.THRESH_BINARY_INV, 15, 5)

contours, _ = cv2.findContours(otsu_thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APP
contour_vis = img6.copy()
cv2.drawContours(contour_vis, contours, -1, (0, 0, 255), 2)

print(f"Found {len(contours)} separate regions (foreground blobs).")
for i, c in enumerate(contours):
    area = cv2.contourArea(c)
    x, y, w, h = cv2.boundingRect(c)
    print(f"  Region {i}: area={area:.0f}px, bounding box=({x},{y},{w},{h})"

show([img6, otsu_thresh, adaptive_thresh, contour_vis],
     ["Original", "Otsu Threshold", "Adaptive Threshold", "Contours"], figsi
```

```
Found 2 separate regions (foreground blobs).
  Region 0: area=14400px, bounding box=(230,90,121,121)
  Region 1: area=15176px, bounding box=(50,80,141,141)
```

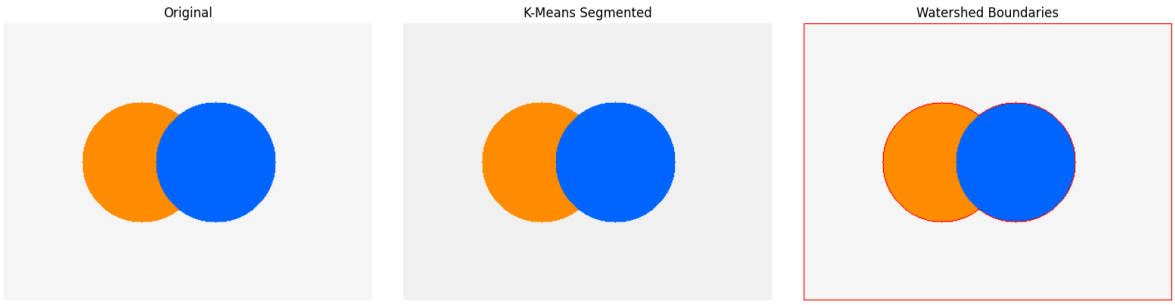


7. Segmentation — K-Means Clustering & Watershed *(Mid-Level Vision)*

- **K-Means**: groups pixels by color similarity (unsupervised clustering)
- **Watershed**: treats the image like a topographic map to separate touching/overlapping objects

```
def create_sample_image_7():
    img = np.full((300, 400, 3), (245, 245, 245), dtype=np.uint8)
    cv2.circle(img, (150, 150), 65, (0, 140, 255), -1)
    cv2.circle(img, (230, 150), 65, (255, 100, 0), -1)  # overlapping circle
    return img

def kmeans_segmentation(img, k=3):
    data = img.reshape((-1, 3)).astype(np.float32)
    criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 20, 1.0)
    _, labels, centers = cv2.kmeans(data, k, None, criteria, 10, cv2.KMEANS_
    centers = np.uint8(centers)
    return centers[labels.flatten()].reshape(img.shape)

def watershed_segmentation(img):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    _, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRE
    kernel = np.ones((3, 3), np.uint8)
    opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations=2)
    sure_bg = cv2.dilate(opening, kernel, iterations=3)
    dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
    _, sure_fg = cv2.threshold(dist_transform, 0.4 * dist_transform.max(), 2
    sure_fg = np.uint8(sure_fg)
    unknown = cv2.subtract(sure_bg, sure_fg)
    _, markers = cv2.connectedComponents(sure_fg)
    markers = markers + 1
    markers[unknown == 255] = 0
    markers = cv2.watershed(img.copy(), markers)
    result = img.copy()
    result[markers == -1] = [0, 0, 255]
    return result, markers

img7 = create_sample_image_7()
kmeans_result = kmeans_segmentation(img7, k=3)
watershed_result, markers = watershed_segmentation(img7)
n_objects = max(len(np.unique(markers)) - 2, 0)

print("K-Means grouped pixels into 3 color clusters.")
print(f"Watershed separated the touching circles into ~{n_objects} distinct

show([img7, kmeans_result, watershed_result],
     ["Original", "K-Means Segmented", "Watershed Boundaries"], figsize=(16,
```

```
K-Means grouped pixels into 3 color clusters.
Watershed separated the touching circles into ~1 distinct objects.
```



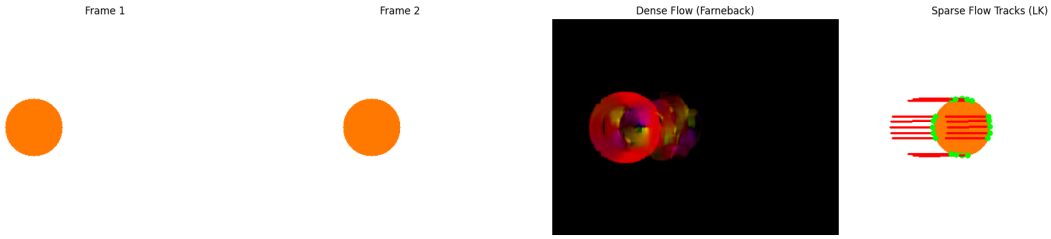
---

8. Motion Estimation — Optical Flow *(Mid-Level Vision)*

Estimate how pixels/objects move between two consecutive frames — another mid-level building block used for tracking and video understanding. We synthesize two frames where a shape moves, then compute dense (Farneback) and sparse (Lucas-Kanade) optical flow.

```
def make_frame(circle_center):
    img = np.full((300, 400, 3), (255, 255, 255), dtype=np.uint8)
    cv2.circle(img, circle_center, 40, (0, 120, 255), -1)
    return img

frame1 = make_frame((100, 150))
frame2 = make_frame((160, 150))  # moved 60px right

gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

flow = cv2.calcOpticalFlowFarneback(gray1, gray2, None, 0.5, 3, 15, 3, 5, 1.
magnitude, angle = cv2.cartToPolar(flow[..., 0], flow[..., 1])
hsv = np.zeros_like(frame1)
hsv[..., 1] = 255
hsv[..., 0] = angle * 180 / np.pi / 2
hsv[..., 2] = cv2.normalize(magnitude, None, 0, 255, cv2.NORM_MINMAX)
flow_vis = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)
```

```
p0 = cv2.goodFeaturesToTrack(gray1, maxCorners=20, qualityLevel=0.3, minDist
p1, status, _ = cv2.calcOpticalFlowPyrLK(gray1, gray2, p0, None)

lk_vis = frame2.copy()
for new, old, st in zip(p1, p0, status):
    if st[0] == 1:
        a, b = new.ravel(); c, d = old.ravel()
        cv2.line(lk_vis, (int(c), int(d)), (int(a), int(b)), (0, 0, 255), 2)
        cv2.circle(lk_vis, (int(a), int(b)), 4, (0, 255, 0), -1)

show([frame1, frame2, flow_vis, lk_vis],
     ["Frame 1", "Frame 2", "Dense Flow (Farneback)", "Sparse Flow Tracks (L

print("Red lines in the last panel show each tracked point's motion vector f
```




```
Red lines in the last panel show each tracked point's motion vector frame1 ->
```

---

9. Object Recognition / Classification *(High-Level Vision)*

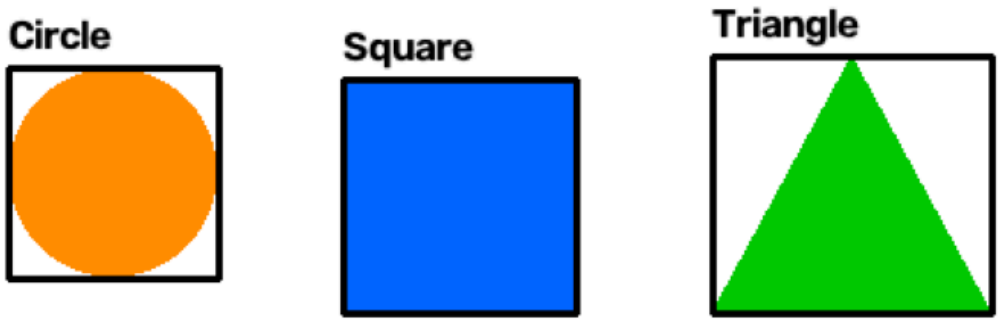
Go from "regions and features" to an actual semantic label — answering "what is this?" rather than just "where are the edges?" via a rule-based shape classifier (polygon approximation + circularity).

```
def create_scene_9():
    img = np.full((300, 500, 3), (255, 255, 255), dtype=np.uint8)
    cv2.circle(img, (80, 80), 45, (0, 140, 255), -1)
    cv2.rectangle(img, (180, 40), (280, 140), (255, 100, 0), -1)
    pts = np.array([[400, 30], [460, 140], [340, 140]], np.int32)
    cv2.fillPoly(img, [pts], (0, 200, 0))
    return img
```

```
def classify_shape(contour):
    perimeter = cv2.arcLength(contour, True)
    approx = cv2.approxPolyDP(contour, 0.03 * perimeter, True)
    area = cv2.contourArea(contour)
    if len(approx) == 3:
        return "Triangle"
    elif len(approx) == 4:
        x, y, w, h = cv2.boundingRect(approx)
        aspect_ratio = w / float(h)
        return "Square" if 0.9 <= aspect_ratio <= 1.1 else "Rectangle"
    else:
        circularity = 4 * np.pi * area / (perimeter ** 2 + 1e-5)
        return "Circle" if circularity > 0.7 else "Unknown"

img9 = create_scene_9()
gray9 = cv2.cvtColor(img9, cv2.COLOR_BGR2GRAY)
_, thresh9 = cv2.threshold(gray9, 240, 255, cv2.THRESH_BINARY_INV)
contours9, _ = cv2.findContours(thresh9, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX

result9 = img9.copy()
print(f"Detected {len(contours9)} objects. Classifying each:\n")
for c in contours9:
    label = classify_shape(c)
    x, y, w, h = cv2.boundingRect(c)
    cv2.rectangle(result9, (x, y), (x + w, y + h), (0, 0, 0), 2)
    cv2.putText(result9, label, (x, y - 8), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (
    print(f"  Object at ({x},{y}) -> classified as: {label}")

show(result9, "Recognized Shapes", figsize=(8, 5))

print("\nNote: this rule-based classifier stands in for a trained CNN.")
print("In practice, high-level recognition typically uses deep learning")
print("models (e.g. ResNet, YOLO) trained on large labeled datasets.")
```

```
Detected 3 objects. Classifying each:

  Object at (180,40) -> classified as: Square
  Object at (35,35) -> classified as: Circle
  Object at (340,30) -> classified as: Triangle
```

R




```
Note: this rule-based classifier stands in for a trained CNN.
In practice, high-level recognition typically uses deep learning
models (e.g. ResNet, YOLO) trained on large labeled datasets.
```

---

10. Full Pipeline — Acquisition to Decision *(Low → Mid → High Level)*

Chain everything together into one end-to-end system:

1. Image Acquisition
2. Preprocessing
3. Feature Extraction (edges)
4. Segmentation
5. Representation & Description (shape/color descriptors)
6. Recognition/Interpretation (classify regions)
7. Decision Making (act on what was recognized)

Scenario: a simple "quality control" system scans a scene and decides ACCEPT/REJECT based on whether it finds a red "defect" marker.

```
def step1_acquire():
    img = np.full((300, 500, 3), (255, 255, 255), dtype=np.uint8)
    cv2.circle(img, (100, 150), 50, (0, 180, 0), -1)
    cv2.rectangle(img, (220, 90), (330, 210), (0, 180, 0), -1)
    cv2.circle(img, (430, 150), 45, (0, 0, 220), -1)  # red = defect
    return img

def step2_preprocess(img):
    return cv2.GaussianBlur(img, (5, 5), 0)

def step3_extract_features(img):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    return cv2.Canny(gray, 50, 150)
```